Workgroup: Network Management Research Group
Internet-Draft: draft-cui-nmrg-auto-test-02
Published: 29 May 2026
Intended Status: Informational
Expires: 30 November 2026
Authors:
Y. Cui
Tsinghua University
Y. Wei
Tsinghua University
K. Chi
Tsinghua University
X. Xie
Tsinghua University
S. Prabhu
Nokia

A Framework for AI-Assisted Network Protocol Testing from Specifications

Abstract

This document describes a framework for AI-assisted network protocol testing from protocol specifications. The framework is organized around the workflow that connects specification text, structured protocol representation, test generation, executable tester and Device Under Test (DUT) artifacts, and execution feedback.

The central problem addressed by this document is how to preserve test-relevant protocol semantics as a testing workflow moves from natural-language specifications to executable tests. The document discusses design motivations, trade-offs, human review points, illustrative cases, and open research challenges for this workflow. It does not define a protocol, a test case format, or a maturity-level classification.

The RFC Editor will remove this note

About This Document

This note is to be removed before publishing as an RFC.

The latest revision of this draft can be found at https://datatracker.ietf.org/doc/draft-cui-nmrg-auto-test/. Status information for this document may be found at https://datatracker.ietf.org/doc/draft-cui-nmrg-auto-test/.

Discussion of this document takes place on the NMRG Research Group mailing list (mailto:nmrg@ietf.org), which is archived at https://datatracker.ietf.org/rg/nmrg/. Subscribe at https://www.ietf.org/mailman/listinfo/nmrg/.

Source for this draft and an issue tracker can be found at https://github.com/WheaterW/draft-cui-nmrg-auto-test.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 30 November 2026.

Copyright Notice

Table of Contents

1. Introduction

Network protocol testing is used to validate whether an implementation behaves according to the semantics defined by protocol specifications. Such testing is required during device development, interoperability validation, procurement evaluation, regression testing, and operational troubleshooting.

Traditional protocol testing remains largely manual. Engineers read long specifications, identify test points, design topologies, write test procedures, create DUT configurations and tester scripts, execute tests, and inspect results. Model-based approaches can automate parts of this process, but they often depend on protocol-specific models that are expensive to build and difficult to reuse across new or rapidly evolving protocols.

Recent advances in artificial intelligence, especially Large Language Models (LLMs), create new opportunities for automating parts of this workflow. However, protocol testing is not a generic text-to-code task. A protocol test begins with requirements distributed across specification text and often ends as a coordinated set of tester actions, DUT configurations, timing assumptions, packet observations, and result oracles. Small losses of protocol semantics across these stages can create invalid tests or misleading failure reports.

This document therefore focuses on a framework question: how can a testing workflow carry test-relevant information from protocol specifications to executable test artifacts while remaining auditable and controllable? The framework decomposes the workflow into structured protocol representation, test generation, executable artifact generation, test execution, and feedback-based refinement. At each stage boundary, the important handoff is not only the generated artifact, but also the protocol semantics, assumptions, and review points that are passed to the next stage.

2. Problem Scope and Assumptions

This document focuses on specification-derived protocol testing. The primary input is a protocol specification, such as an RFC or a related update document. The target outputs are test cases, tester scripts, DUT configurations, execution results, and reports that can be traced back to the specification.

The framework is mainly intended for conformance, functional, robustness, regression, and related protocol-behavior tests. It does not attempt to cover all implementation-specific defects. Bugs in local management channels, proprietary command-line behavior, vendor-specific configuration subsystems, or non-standard extensions can require additional input documents such as vendor manuals, YANG models, implementation notes, or operational policies.

The framework also assumes that AI-assisted components are not trusted as fully autonomous oracles. Human review remains important at workflow boundaries, especially for structured representation inspection, test oracle validation, executable artifact validation, and final bug confirmation. The intended role of automation is to reduce repetitive expert effort while preserving traceability and expert control.

3. Definitions and Acronyms

DUT: Device Under Test

Tester: A network device implementing multiple network protocols to support protocol conformance and performance testing. It generates test-specific packets or traffic, emulates target network behaviors, and analyzes received packets to evaluate protocol compliance and performance.

LLM: Large Language Model

FSM: Finite State Machine

API: Application Programming Interface

CLI: Command Line Interface

Test Case: A specification of conditions and inputs to evaluate a protocol behavior.

Tester Script: An executable program or sequence of instructions that controls a protocol tester to generate test traffic, interact with the DUT according to a specified test case, and collect relevant observations for result evaluation.

4. Network Protocol Testing Scenarios

Network protocol testing is required in many scenarios. This document outlines two common phases where protocol testing plays a critical role:

1. Device Development Phase: During the development of network equipment, vendors need to ensure that their devices conform to protocol specifications. This requires the construction of a large number of test cases. Testing during this phase can involve both protocol testers and the DUT, or it can be performed solely through interconnection among DUTs.
2. Procurement Evaluation Phase: In the context of equipment acquisition by network operators or enterprises, candidate equipment suppliers need to demonstrate compliance with specified requirements. In this phase, third-party organizations typically perform the testing to ensure neutrality. This type of testing is usually conducted as black-box testing, requiring the use of protocol testers interconnected with the DUT. The test cases are executed while observing whether the DUT behaves in accordance with expected protocol specifications.

5. Key Elements of Network Protocol Testing

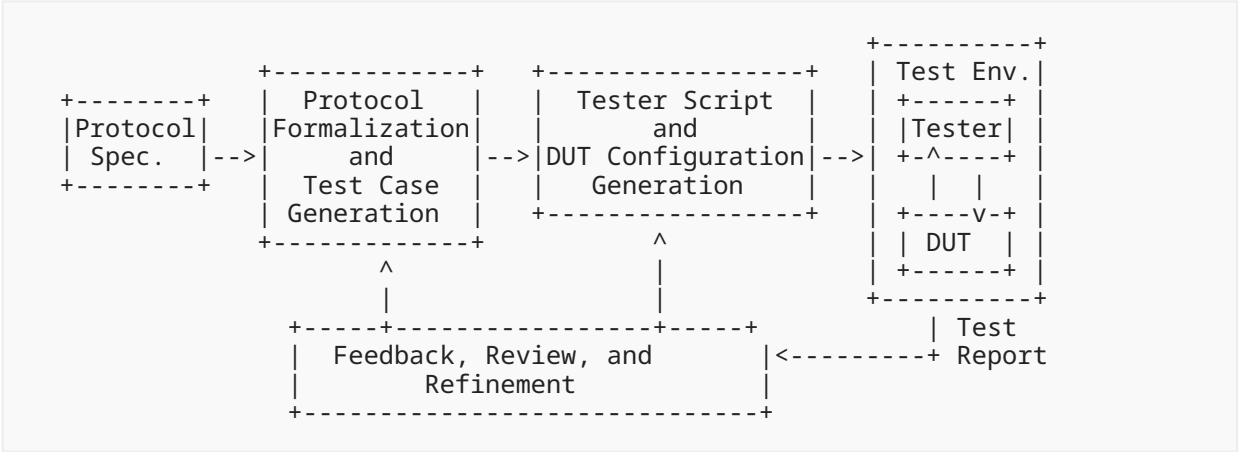
Network protocol testing is a complex and comprehensive process that typically involves multiple parties and various necessary components. The following entities are generally involved in protocol testing:

1. DUT: The DUT can be a physical network device (such as switches, routers, and firewalls) or a virtual network device (such as FRRouting (FRR) routers and others).
2. Tester: A protocol tester is a specialized network device that usually implements a standard and comprehensive protocol stack. It can generate test traffic, collect and analyze incoming traffic, and produce test results. Protocol testers can typically be controlled via scripts, allowing automated interaction with the DUT to carry out protocol tests.
3. Test Cases: Protocol test cases can cover various categories, including protocol conformance tests, functional tests, and performance tests. Each test case typically includes essential elements such as test topology, step-by-step procedures, and expected results. A well-defined test case also includes detailed configuration parameters.
4. Test Topology: Each test case needs to specify the network topology it requires. Before executing a test case, the corresponding topology needs to be established accordingly. In a batch testing scenario, frequent changes in topology can be time-consuming and inefficient. To mitigate this overhead, it is common practice to construct a minimal common topology that satisfies the requirements of all test cases in a given batch. This minimizes the number of devices and links needed while ensuring that each test case can be executed within the shared topology.

5. DUT Configuration: Before executing a test case, the DUT needs to be initialized with specific configurations according to the test case requirements (setup). Throughout the test, the DUT configuration can undergo multiple modifications as dictated by the test scenario. Upon test completion, appropriate configurations are usually applied to restore the DUT to its initial state (teardown).
6. Tester Configuration and Scripts: In test scenarios involving protocol testers, the tester often plays the active role by generating test traffic and orchestrating the test process. This requires the preparation of both tester-specific configurations and execution scripts. Tester scripts are typically designed in coordination with the DUT configurations to ensure proper interaction during the test.

6. Framework Overview

A typical AI-assisted network protocol testing framework is illustrated as follows.



The framework has five stages:

1. Structured protocol representation: transform specification text into a test-relevant representation.
2. Test case generation: derive test points, templates, parameters, and oracles.
3. Executable artifact generation: translate abstract tests into coordinated tester scripts and DUT configurations.
4. Test execution: run the generated artifacts in a controlled test environment.
5. Feedback and refinement: analyze failures and decide whether to refine tests, artifacts, or bug reports.

The output of each stage becomes a handoff to the next stage. For this reason, each stage can expose not only generated content, but also source references, assumptions, constraints, and review status.

6.1. Structured Protocol Representation

Protocol specifications are usually written for human readers. They include message formats, state machines, timers, variables, algorithms, exception handling, and normative behavior spread across sections and sometimes across multiple update documents. Directly generating executable tests from such text risks losing details that determine whether a test is valid.

The framework therefore introduces a structured protocol representation as an intermediate artifact between specification text and downstream test generation. The representation is not required to be a complete formal model of the protocol. Instead, it preserves the protocol semantics that are relevant for testing.

Such a representation can include:

- message formats, fields, constraints, and valid or invalid values
- local data structures, timers, counters, and protocol variables
- state machines and state transitions
- event-action rules and packet processing behavior
- protocol algorithms, such as route selection or path computation
- error handling and exception behavior
- relationships among modules, such as which messages trigger which transitions or algorithms
- links back to source specification sections

A practical construction workflow can be organized into three steps.

First, semantic enrichment organizes the specification into sections, subsections, cross-references, normative statements, summaries, and update relationships. This step can combine rule-based extraction with AI-assisted summarization.

Second, module induction groups related text into protocol modules, such as message formats, state machines, algorithms, and event-action rules. Each module remains linked to the source text used to construct it.

Third, formalization serializes each module into a structured representation that can be queried, traversed, reviewed, and used by test generation tools.

Protocol updates require special handling. Update documents often add, modify, or deprecate specific protocol behavior rather than restating a complete protocol. An update-aware representation identifies update points, maps them to existing modules, and expresses the update as a differential change to the base representation.

The main design trade-off is between completeness and usefulness. A fully formal protocol model can be expensive to build and difficult to apply across many protocols. A test-relevant representation is less ambitious, but can be more practical if it preserves the semantics needed to generate valid tests, derive oracles, and trace failures back to specifications.

6.2. Test Case Generation

Once a structured protocol representation is available, test generation identifies test points and expands them into test cases. Test points can be derived from normative statements, message constraints, state transitions, algorithms, error handling requirements, and update deltas.

The framework separates test templates from test parameters.

A test template captures the reusable structure of a test:

- test objective
- specification reference
- test topology
- preconditions and static configuration
- test procedure
- expected result or oracle
- variable placeholders

Parameters instantiate those placeholders with concrete values. Parameter generation can include valid values, invalid values, boundary values, timing values, topology variants, and values computed by helper logic. Oracle values can also require computation, especially when expected behavior depends on route selection, timers, path attributes, or other protocol algorithms.

This separation is a key design choice. Templates provide breadth across protocol functions and test scenarios. Parameters provide depth across value spaces and boundary conditions. Equivalence partitioning or similar reduction techniques can then group parameter combinations that exercise the same protocol behavior, reducing redundant execution without discarding meaningful coverage.

Generated test cases still require review. Correctness means that a test reflects the intended protocol semantics. Coverage means that the test suite exercises relevant protocol functions, behaviors, and parameter spaces. Because protocol test cases mix natural language, topology assumptions, configuration fragments, packet observations, and oracles, systematic evaluation of correctness and coverage remains an open research challenge.

6.3. Executable Artifact Generation and Feedback

Executable artifact generation translates abstract test cases into runnable tester scripts and DUT configurations. This step is difficult because tester actions and DUT configurations are coordinated together. They agree on topology, addresses, protocol parameters, timing, expected packets, and oracle logic.

Directly exposing heterogeneous tester APIs and device-specific configuration mechanisms creates a large and error-prone action space. A useful framework can therefore introduce explicit testbed abstractions, such as:

- a DUT control abstraction for applying configuration, changing runtime state, and collecting diagnostic outputs
- a tester-side logic abstraction for protocol emulation, traffic control, packet capture, and result queries
- an execution abstraction for running tests, collecting logs, and reporting pass/fail outcomes

These abstractions constrain the generation problem and make artifacts easier to validate. Validation covers both syntax and semantics. Syntax validation detects invalid API calls or CLI commands. Semantic validation checks whether the artifacts implement the intended test logic and whether tester and DUT configurations are mutually consistent.

Execution feedback requires careful interpretation. A failed test does not always indicate a protocol implementation defect. It can be caused by an invalid test case, an incorrect oracle, a generated script error, a DUT configuration issue, a topology problem, or environmental instability. Feedback analysis distinguishes at least four categories:

- likely DUT protocol violation
- invalid or ambiguous test case
- executable artifact error
- test environment or setup failure

A feedback record can be organized into observed inputs, analysis, and refinement actions. Observed inputs can include pass/fail results, execution logs, diagnostic outputs, telemetry collected during the run, and DUT state or configuration snapshots. The analysis maps discrepancies between expected and observed behavior to likely causes, while refinement actions can correct the test case, adjust parameters or timing, update the oracle, or fix tester scripts and DUT configurations.

Human review is most valuable at the boundaries where these categories are decided. In practical deployments, the goal is not to remove experts from the workflow, but to shift their effort from manual artifact construction to targeted review, correction, and final decision-making.

7. Design Considerations and Trade-offs

7.1. End-to-End Prompting vs. Structured Workflow Boundaries

End-to-end prompting can produce useful drafts, but it is a weak workflow boundary for protocol testing. It often hides which specification text was used, which constraints were preserved, and which assumptions were introduced. A structured representation provides a more inspectable boundary between specification analysis and test generation.

7.2. Test-Relevant Representation vs. Complete Formal Model

Complete formal models can provide strong guarantees, but they are expensive to build and difficult to maintain across many protocols and update documents. A test-relevant representation makes a weaker claim: it aims to preserve the semantics needed for testing. This is less complete, but more feasible for broad protocol coverage.

7.3. Template Breadth vs. Parameter Depth

Generating many concrete tests directly can produce large and redundant suites. Separating templates from parameters allows broad functional coverage and deep parameter exploration to scale independently. The cost is that the framework also manages parameter reduction, oracle computation, and test-suite scheduling.

7.4. Feedback Automation vs. Human Confirmation

Execution feedback is necessary because some errors only appear at runtime. However, automatic failure classification can be misleading. The framework keeps experts in the loop for high-impact decisions such as accepting a generated oracle, approving executable artifacts, and confirming protocol violations.

7.5. Specification Scope vs. Implementation Scope

Specification-derived testing targets behavior defined by protocol specifications. It can miss implementation-specific bugs in management functions, local control channels, vendor extensions, or configuration subsystems. Expanding the input corpus can broaden coverage, but also introduces new ambiguity and source-trust questions.

8. Cases and Experience

This section gives illustrative cases that motivate the framework. The cases are not intended to define required behavior for all implementations.

8.1. Case 1: Update-Aware Testing

Protocol behavior changes over time. For example, an update document can deprecate a message field, add a timer, or modify a state-machine transition. A framework that treats the update document in isolation can miss the affected base-protocol behavior.

An update-aware representation can map an update to the corresponding base modules, such as a BGP path attribute or state-machine rule. Test generation can then focus on the changed behavior. For example, when a path segment is deprecated, the generated test can advertise a route containing that segment and check whether the DUT handles or propagates it according to the updated specification.

The lesson is that update documents are best represented as differential changes over existing protocol modules, not as standalone specifications.

8.2. Case 2: Specification-Derived Bug Exposure

A specification can define behavior that is easy to overlook in manual test suites. For example, a routing protocol can define how a default route is expected to be originated, advertised, or accepted. A generated test can configure the relevant condition, capture protocol updates, and check whether the expected route appears.

The lesson is that systematic extraction of test points from structured protocol behavior can expose gaps in manually curated test suites, especially for less frequently tested edge cases.

8.3. Case 3: Parameterized Boundary Testing

Some bugs appear only under specific parameter relationships rather than under a single fixed input. For example, a state-machine or route-selection behavior can depend on relative priority values, timer ordering, prefix relationships, or topology-dependent reachability.

Template-parameter separation supports this class of tests. A template captures the protocol scenario, while parameter instantiation explores relevant value relationships. Equivalence partitioning can reduce redundant combinations while preserving representative boundary cases.

The lesson is that test depth often depends on parameter relationships and oracle computation, not only on the number of test cases.

8.4. Implementation Experience

Prototype implementations and deployment-oriented studies suggest that the framework can reduce repetitive expert effort while preserving useful protocol-test coverage. Early experience across multiple routing protocols indicates that structured representation, template-parameter separation, and execution feedback can help generate large test suites, reduce redundant parameter instances, and expose defects not covered by manually curated suites.

These observations are non-normative implementation experience, not a general guarantee or a requirement for conforming systems. Detailed experimental methodology and quantitative results are out of scope for this document and can be reported separately. The experience mainly supports the need for structured workflow boundaries, review points, and test-suite management.

9. Research Challenges

Several research challenges remain open for AI-assisted protocol testing.

Representation fidelity: The community lacks widely accepted metrics for whether a structured protocol representation preserves the semantics needed for testing.

Test case correctness and coverage: Protocol test cases combine natural language, topology, configuration, packets, and oracles, making correctness and coverage hard to measure systematically.

Update-aware testing: RFC updates, extensions, and deprecations require methods to localize changes and propagate them to affected tests.

Cross-vendor artifact abstraction: Tester APIs and device configuration mechanisms differ across vendors and testbeds. Common abstractions are needed to make generated artifacts portable and reviewable.

Oracle generation and validation: Some expected results can be computed from protocol algorithms, while others require judgment or implementation-specific context.

Failure triage: A failed test can indicate a DUT bug, invalid test case, script error, configuration error, or environment issue. Reliable triage remains difficult.

Feedback-loop reproducibility: Refinement decisions need enough recorded context to be replayed, audited, and compared across tool versions or test environments.

Test-suite management: Large generated suites require reduction, scheduling, topology clustering, prioritization, and regression selection.

Auditability and traceability: Test results need traceability back to specification text, representation modules, generated test cases, parameters, and executable artifacts.

Safety and security: Automatically generated code and configuration can disrupt test environments or create misleading reports. Sandboxing, validation, and human approval remain necessary for high-impact operations.

10. Security Considerations

1. Execution of Unverified Generated Code: Automatically generated test scripts or configurations (e.g., CLI commands, tester control scripts) can include incorrect or harmful instructions that misconfigure devices or disrupt test environments. Mitigation: Validate all generated artifacts, including syntax checking, semantic verification against protocol constraints, and dry-run execution in sandboxed environments.
2. AI-Assisted Component Risks: LLMs can produce incorrect or insecure outputs due to their probabilistic nature or prompt manipulation. Mitigation: Apply input sanitization, prompt hardening, and human-in-the-loop validation for critical operations.

11. IANA Considerations

This document has no IANA actions.

Acknowledgments

This work is supported by the National Key R&D Program of China.

Contributors

Zhen Li
Beijing Xinertel Technology Co., Ltd.
Email: lizhen_fz@xinertel.com

Zhanyou Li
Beijing Xinertel Technology Co., Ltd.
Email: lizy@xinertel.com

Authors' Addresses

Yong Cui
Tsinghua University
Email: cuiyong@tsinghua.edu.cn

Yunze Wei
Tsinghua University
Email: wyz23@mails.tsinghua.edu.cn

Kaiwen Chi
Tsinghua University
Email: ckw24@mails.tsinghua.edu.cn

Xiaohui Xie
Tsinghua University
Email: xiexiaohui@tsinghua.edu.cn

Shailesh Prabhu
Nokia
Email: shailesh.prabhu@nokia.com